Informe APE2_Parte 2

1. Datos Generales

Asignatura	Software Security
Ciclo	8 A
Unidad	1
Resultado de Aprendizaje de la Unidad	R1. Diseña e implementa software bajo los principios básicos de estándares seguros, en el marco de los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2 (Fases 1 a 4)
Título	Auditoría técnica y endurecimiento de la aplicación
Docente	Valeria Herrera Salazar
Integrantes	Dalton Maza, Leonardo Peralta, Gabriel Espinoza
Período	Del 26 de octubre al 27 de noviembre de 2026 (Semanas 5 a 8)
Horario	Jueves: 09h30 – 11h30 Viernes: 07h30 – 09h30
Lugar	Aula de Clase
Tiempo planificado	16 horas (4 horas por semana)

Estructura de un token JWT (cabecera, payload y firma), riesgos del algoritmo none y de los tokens sin expiración, concepto de referencias directas inseguras a objetos

Json Web Token es un estándar que define un formato autónomo y compacto para la transmisión de información entre partes como un objeto JSON y se constituye de las siguientes partes:
Cabecera - Header: Objeto JSON que consta de 2 partes, el tipo token (typ que suele JWT) y el algoritmo de firma utilizado (alg, como HS256, RS256). Consecuentemente este JSON es codificado en Base64URL.

Cuerpo - Payload: Alberga declaraciones (claims) que se refiere a la información que se busca transmitir, se tiene 3 tipos de claims (iss, exp, sub) de categoria pública o privada que es definida por la aplicación presente (role, id), cabe destacar que el payload se encuentra codificado y no cifrado por ende cualquiera puede leerlo

Firma - Signature: Es generado empleando la cabecera codificada, el payload codificado, un secreto o también una clave privada y empleando el algoritmo criptográfico que se define en el header que tiene como función principal el garantizar la integridad del token en caso de ser modificado y la autenticidad del emisor.

Para casos donde el token no requiere firma se permite el uso del algoritmo “none” para escenarios donde existen token que albergan información pública no sensible, pero representa un riesgo de seguridad en caso de que el servidor no esté bien configurado en donde el atacante podría interceptar un token válido, hacer la decodificación del payload y manipular los claims existentes para luego volver a codificar el token cambiando el header a “none” y eliminar la firma; un servidor podría aceptar el token modificado como válido otorgando privilegios no autorizados si las librerías de validación no tienen especificado qué algoritmo esperar, en este caso aceptando el none que el atacante ha cambiado.

Otro aspecto necesario es el tiempo de expiración el cual deber ser limitado y no ser excesivamente largo, el infractor podría suplantar la identidad de la víctima durante todo el tiempo de vida que dure dicho token.

Insecure Direct Object Reference (IDOR): Se produce cuando una aplicación expone una referencia directa a un objeto interno (IDs de base de datos, claves) en un tipo de parámetro (URL, formularios) y no realizar una verificación adecuada de autorización para asegurar que el usuario que hace la solicitud al objeto tiene permiso para su acceso. El atacante podría modificar el token en caso de lograr saltarse la firma, o si el servidor confía ciegamente en el ID del token o simplemente cambiar el ID en la URL, para acceder a datos de otros usuarios, Por lo tanto la autorización debe ser siempre validada en el lado del servidor y no se debe confiar en que el cliente (el token) diga qué recursos puede ver.

Actividad 2: Laboratorio de análisis de tokens (80 minutos)

Cada pareja captura un token de sesión de su aplicación (o del entorno controlado), lo decodifica con jwt.io o CyberChef, y analiza sus claims, el algoritmo de firma y la expiración, documentando los hallazgos con evidencia.

Decodificación del JWT

Header (algoritmo y tipo):

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload (claims):

```
{
  "id": "6ad947fb990dc093ce1339d0",
  "role": "Administrador",
  "iat": 1794496881,
  "exp": 1795101681
}
```

Firma: 8PjiI24svluGXC704OZREsHMHjQLLkD9lqdzVCS3nJg — HMAC-SHA256.

Traducción de timestamps (UTC):

- iat (emitido): 1794496881 → 12 de noviembre de 2026, ~15:21 UTC
- exp (expira): 1795101681 → 19 de noviembre de 2026, ~15:21 UTC
- Duración de la sesión: 604 800 segundos = 7 días exactos

Hallazgos

ID	Control evaluado	Evidencia	Resultado	Riesgo preliminar
A1	Algoritmo de firma	alg: HS256 (simétrico, no none)	Cumple	Bajo
A2	Exposición de datos sensibles en claims	Solo id y role; sin email/clave	Cumple	Bajo

A3	Tiempo de expiración	exp - iat = 604800 s (7 días)	No cumple	Alto
A4	Mecanismo de revocación	Sin jti; no hay evidencia de lista de revocación/blacklist	No cumple	Alto
A5	Rol embebido sin claims de contexto	role: "Administrador" sin iss/aud; depende de que el servidor revalide el rol en cada endpoint	Parcial	Alto

Tabla de registro de hallazgos:

Hallazgo	Evidencia	Riesgo preliminar	Acción correctiva
A3 — Expiración de sesión excesiva	iat=1794496881, exp=1795101681 (7 días)	Alto	Reducir exp a 15 min + implementar refresh token con rotación
A4 — Sin revocación de tokens	No existe jti ni blacklist visible en el backend	Alto	Añadir jti único y tabla/caché de tokens revocados (logout, cambio de contraseña)
A5 — Rol de alto privilegio embebido en el token sin re-verificación documentada	Claim role: "Administrador"	Alto	Verificar en servidor (no confiar en el claim del cliente) + probar IDOR entre usuarios con distinto rol en Sesión 2

Revisión de los controles de referencia

Control	¿Se cumple en la app?	Evidencia
Verificación de autorización por objeto (ownership)	No	PUT/DELETE /data/:id no comparan owner con req.user; DELETE /account/:external no valida ni propiedad ni rol
Expiración de sesión	Parcial (visto en Sesión 1)	Token JWT con exp de 7 días — demasiado largo
Validación de la firma del token en servidor	Sí, pero insuficiente	El middleware (isLoggedIn) válida la firma, pero no valida el rol en operaciones administrativas (POST /role, POST /account con role: "Administrador")

Ejecución de al menos tres pruebas IDOR,verificación del control de roles entre administrador y usuario común, y redacción de la corrección propuesta por cada hallazgo.

Hallazgo	Evidencia	Riesgo	Acción correctiva
IDOR confirmado en DELETE /data/:id	Usuario B (rol Usuario) eliminó el dato de Usuario A — status 200, confirmado en MongoDB	Crítico	Filtrar por { _id: id, owner: req.user._id } en findOneAndDelete (código ya entregado por Copilot)
IDOR confirmado en DELETE /account/:external	Usuario B desactivó la cuenta de Usuario A (state: "INACTIVA")	Crítico	Añadir requireRole("Administrador") antes del controlador
Falta de control de rol en POST /role	Usuario B creó el rol BR76124642, verificado en BD	Alto	Añadir middleware requireRole("Administrador") tras isLoggedIn
Escalada de privilegios en POST /account (sin auth)	Cuenta creada con role: "Administrador" sin ningún token	Crítico	El registro público debe forzar rol "Usuario" server-side; crear ruta separada /account/admin protegida
Escalada de privilegios en POST /account (Usuario B autenticado)	Mismo resultado autenticado como usuario común	Crítico	Misma corrección que el anterior

Hallazgo adicional — Exposición de credenciales en respuestas	El backend devolvió password (hasheado), token y tokenExpiresAt en el JSON de respuesta de cuentas	Alto	Excluir esos campos en la respuesta con .select("-password -token -tokenExpiresAt") o un DTO explícito
Hallazgo de infraestructura (no es IDOR, pero debe anotarse)	El middleware espera el header no estándar Autentication en vez de Authorization	Bajo/Medio	Corregir el nombre del header a Authorization (buena práctica/interoperabilidad); documentar en el informe como desviación de estándar
PUT /data/:id no verificable (endpoint roto)	El validador Joi mezcla req.params con el body y rechaza el campo id con 400, incluso para el Admin	(bloquea el análisis, no es vulnerabilidad de autorización)	Corregir la validación separando req.body/req.query de req.params (código ya entregado); repetir la prueba IDOR en esta ruta después del fix, porque hoy no se pudo confirmar ni descartar

6. Bibliografía:

- Haz Lopez, L. V., et al. (2024). Hacking Ética: Teoría y Prácticas. CID. ISBN: 978-1967344-23-9.
- Vega Briceño, E. (2021). Seguridad de la Información. Editorial Área de Innovación y Desarrollo. ISBN: 978-84-122093-6-5.
- Whitman, M. E., & Mattord, H. J. (2018). Management of Information Security (6th ed.). Cengage Learning.
- Stallings, W. (2020). Cryptography and Network Security: Principles and Practice (8th ed.). Pearson.
- OWASP Foundation. (2025). OWASP Cheat Sheet Series: Password Storage. Recuperado de https://cheatsheetseries.owasp.org/